Model-Based Plot: Linear Regression

Linear Regression

Xu Shirong

School of Economics, Xiamen University

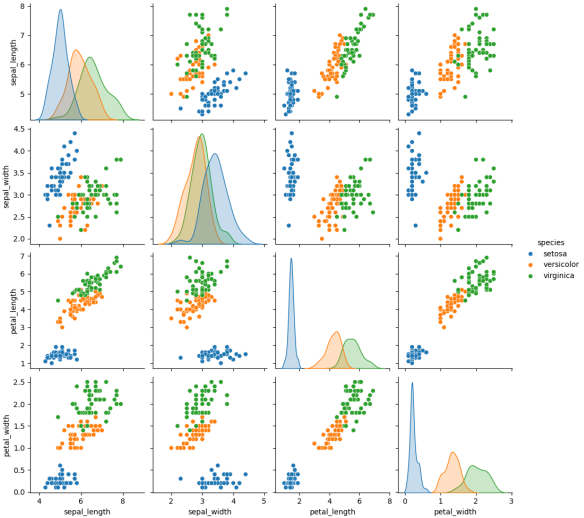
2026

Pairplot: Overview of Multivariate Relationships

```
iris = sns.load_dataset("iris")
sns.pairplot(iris, hue="species")
```

- Pairplots visualize all pairwise relationships.
- Diagonal panels show marginal distributions.
- Color encoding reveals class separation patterns.

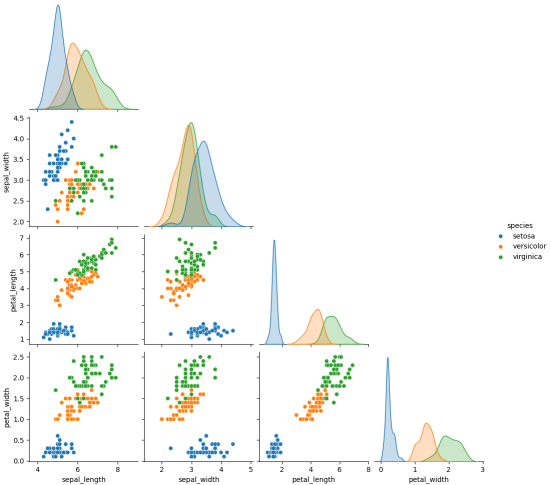
Frame Title



- Pairplots visualize all pairwise relationships.
- Diagonal panels show marginal distributions.
- Color encoding reveals class separation patterns.

Pairplot: Interpreting Separation

```
sns.pairplot(iris, hue="species",corner=True)
```



- Since pairwise plot is symmetric, we can only retain the lower triangle plots.

Correlation Coefficient

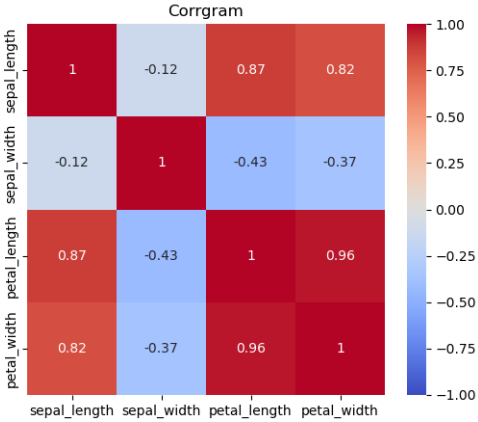
Pearson correlation coefficient:

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

- $\bar{x}, \bar{y}$: sample means of X and Y
- n: number of paired observations
- $r = 0$: no linear relationship
- $r = 1$: perfect positive linear correlation
- $r = -1$: perfect negative linear correlation
- Can you show why this quantity is always in the range $[-1, 1]$?

Correlation Matrix

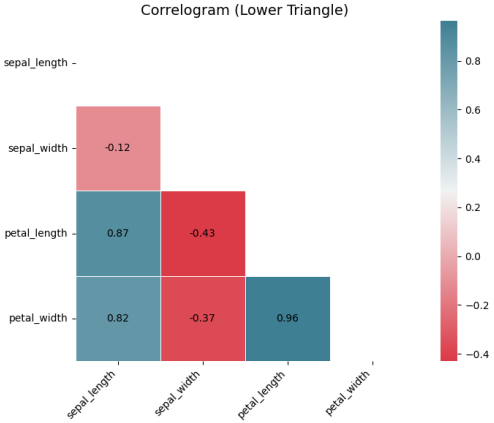
```
corr = iris[["sepal_length","sepal_width","petal_length","petal_width"]].corr()
print(corr)
```



- Correlation measures linear association.
- Values range from -1 to 1.
- Strong correlations suggest redundancy.
- Can you construct an example where two variables are intrinsically correlated but linear correlation is zero.

Correlogram: Lower Triangle

```
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True)
```



- Lower-triangle avoids duplicated entries.
- Easier to scan large correlation matrices.
- Colors encode both magnitude and sign.

Correlation: Teaching Takeaways

- Correlation $\neq$ causation.
- Only captures linear relationships (For example, $X \sim N(0,1)$ and X^2).
- Always visualize before modeling.

Population and Sample

- **Population**: The entire set of objects under study. Treated as a random variable with distribution $F(X)$.
- **Population Distribution**: $F(x)$ with

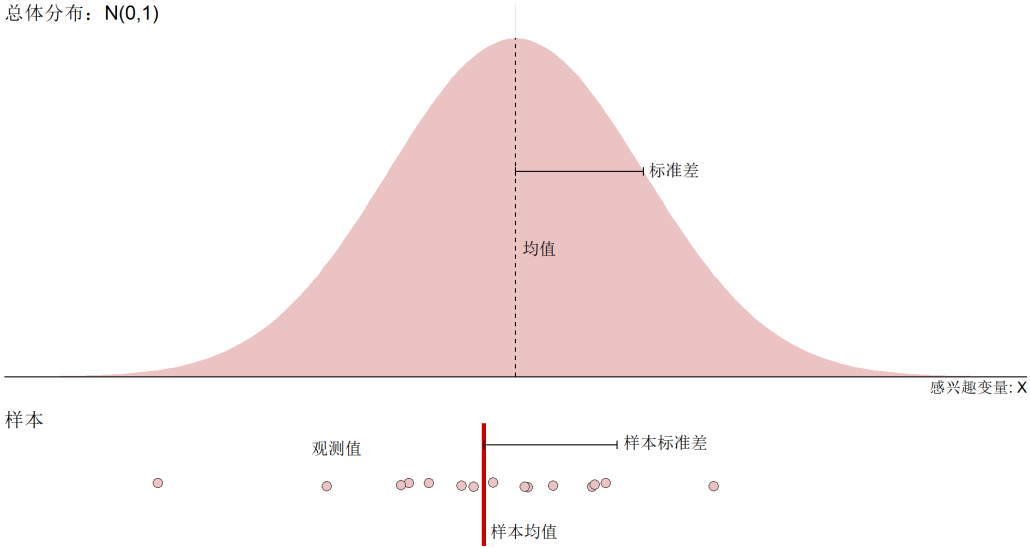
$$\mu = \mathbb{E}[X], \quad \sigma^2 = \mathrm{Var}(X)$$

- **Sampling**: Select a subset from the population to estimate $F(X)$.

$$X_1, X_2, \ldots, X_n$$

  are i.i.d. from $F(X)$.
- This subset is a **sample**, with size n.

Population Distribution - Visualization



Point Estimation

- Use sample statistics to estimate population parameters.
- **Sample Mean** $\bar{X}$ estimates population mean μ:

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

- Expected value and variance:

$$\mathbb{E}(\bar{X}) = \mu, \quad \mathrm{Var}(\bar{X}) = \frac{\sigma^2}{n}$$

Sample Variance and Standard Error

- **Sample Variance**:

$$\hat{\sigma}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2$$

  Why $n-1$?

- **Sample Standard Deviation**: $\hat{\sigma} = \sqrt{\hat{\sigma}^2}$
- **Standard Error of the Mean**:

$$\text{SE} = \frac{\hat{\sigma}}{\sqrt{n}}$$

Properties of Point Estimators

Case 1: Normal Distribution, Known Variance

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right), \quad \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim N(0,1)$$

Case 2: Normal Distribution, Unknown Variance

$$\frac{\bar{X} - \mu}{\hat{\sigma}/\sqrt{n}} \sim t_{n-1}$$

Case 3: Unknown Population, Large Sample (CLT)

$$\frac{\bar{X} - \mu}{\hat{\sigma}/\sqrt{n}} \xrightarrow{d} N(0,1)$$

Summary

- Sample statistics estimate population parameters.
- Key formulas:

$$\bar{X} = \frac{1}{n}\sum X_i, \quad \hat{\sigma}^2 = \frac{1}{n-1}\sum (X_i - \bar{X})^2, \quad \text{SE} = \frac{\hat{\sigma}}{\sqrt{n}}$$

- Choice of distribution (Normal, t, CLT) depends on known information and sample size.

Chocolate Review Dataset - Variables Overview

- **REF** – Unique identifier for each chocolate bar review.
- **Company (Manufacturer)** – Name of the company that produced the chocolate.
- **Company Location** – Location (city/country) of the manufacturer.
- **Review Date** – Date when the chocolate bar was reviewed.
- **Country of Bean Origin** – Country where the cocoa beans were sourced.
- **Specific Bean Origin or Bar Name** – Detailed name of the chocolate bar or specific bean origin.
- **Cocoa Percent** – Cocoa content percentage in the chocolate bar.
- **Ingredients** – List of ingredients in the chocolate bar.
- **Most Memorable Characteristics** – Descriptive flavor notes.
- **Rating** – Reviewer's rating score (numerical, e.g., 1–5 scale).

This dataset contains 2530 observations across 10 variables, capturing both objective data (e.g., cocoa content) and subjective review metrics (e.g., flavor characteristics and rating).

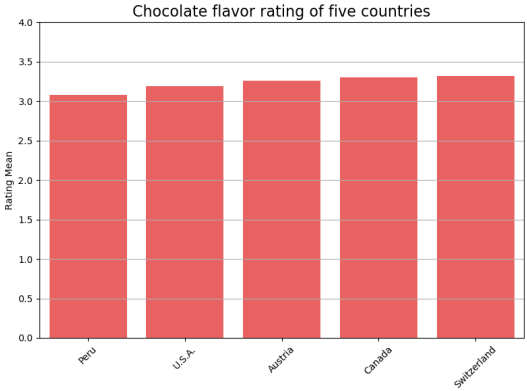
```python
ch_rate = pd.read_csv("chocolate_ratings.csv")
selected_countries = ["U.S.A.", "Switzerland", "Peru", "Canada", "Austria"]
cr_small = ch_rate[ch_rate["Company Location"].isin(selected_countries)]
cr_mean = cr_small.groupby("Company Location").agg(
    mean=("Rating", "mean"),
    sd=("Rating", "std"),
    count=("Rating", "count")
).assign(se=lambda df: df["sd"] / np.sqrt(df["count"])).reset_index()
```

Company Location	mean	sd	count	se
Austria	3.258333	0.324856	30	0.059310
Canada	3.303672	0.416220	177	0.031285
Peru	3.076087	0.545825	23	0.113812
Switzerland	3.318182	0.449042	44	0.067696
U.S.A.	3.190801	0.424297	1136	0.012589

Can we give a ranking over rating mean for these 5 locations?

Bar Plot For Means

```
cr_mean_sorted = cr_mean.sort_values("mean", ascending=True)
plt.figure(figsize=(8, 6))
sns.barplot(data=cr_mean_sorted, x="Company Location", y="mean", color="#FF0000",
    alpha=0.7)
```



- Peru has the lowest rating.
- Switzerland has the highest rating.
- How confident are we in these estimates or in the overall ranking?

Error Bars

- **Purpose:** Used to visualize the uncertainty in parameter estimates.
- **Key point:** It is essential to clearly specify what the error bars represent.

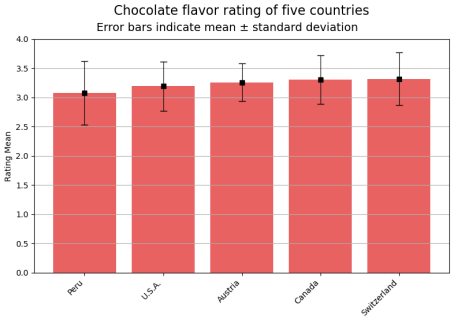
Common types of error bars include:

- Sample mean $\pm$ standard deviation (SD)
- Sample mean $\pm$ standard error (SE)

Note: SD reflects the variability of the data, while SE reflects the uncertainty of the estimated mean.

Bar Plot For Means with SD

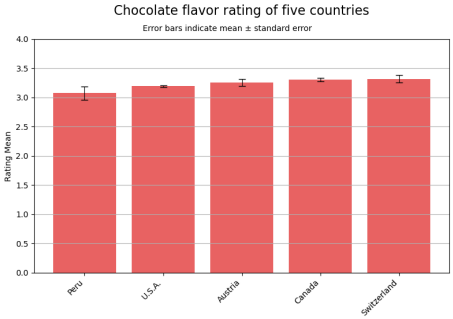
```
sns.barplot(data=cr_mean_sorted, x="Company Location", y="mean", color="#FF0000",
    alpha=0.7, ax=ax)
ax.errorbar(x=np.arange(len(cr_mean_sorted)), y=cr_mean_sorted["mean"],
            yerr=cr_mean_sorted["sd"], fmt="s", color="black", capsize=4, linewidth
                =0.8)
```



- Peru has the lowest rating.
- Switzerland has the highest rating.
- Which location has the lowest variability in rating? (lowest within-group variation.)

Bar Plot For Means with SD

```
fig, ax = plt.subplots(figsize=(8, 6))
sns.barplot(data=cr_mean_sorted, x="Company Location", y="mean", color="#FF0000",
    alpha=0.7, ax=ax)
ax.errorbar(x=np.arange(len(cr_mean_sorted)), y=cr_mean_sorted["mean"],yerr=
    cr_mean_sorted["se"], fmt="none", color="black", capsize=4, linewidth=0.8)
```



- Which location has the lowest estimate uncertainty? (Why?)
- Sample size and Within-group variability (variance)

Common Types of Error Bars

- Sample mean $\pm$ standard deviation (SD)
- Sample mean $\pm$ standard error (SE)
- Confidence interval (CI)

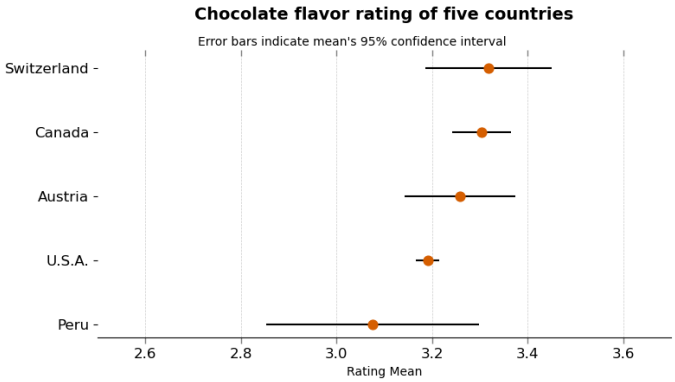
Different error bars reflect different sources of uncertainty. Always specify what your error bars represent.

Line Plot For Means with SE

```
z_critical = 1.96
cr_mean_sorted = cr_mean.sort_values("mean")
locations = cr_mean_sorted["Company Location"]
means = cr_mean_sorted["mean"]
errors = z_critical * cr_mean_sorted["se"]
fig, ax = plt.subplots(figsize=(8, 5))
for i, (mean, err) in enumerate(zip(means, errors)):
    ax.hlines(y=i, xmin=mean - err, xmax=mean + err, color="black", linewidth=1.5)
ax.scatter(means, np.arange(len(means)), color="#D55E00", s=60, zorder=3)
```

What are we doing in the above codes?

Line Plot For Means with SE



- This figure is more illustrative for summarizing the uncertainty of 5 mean estimates.
- Interpretation of confidence interval.

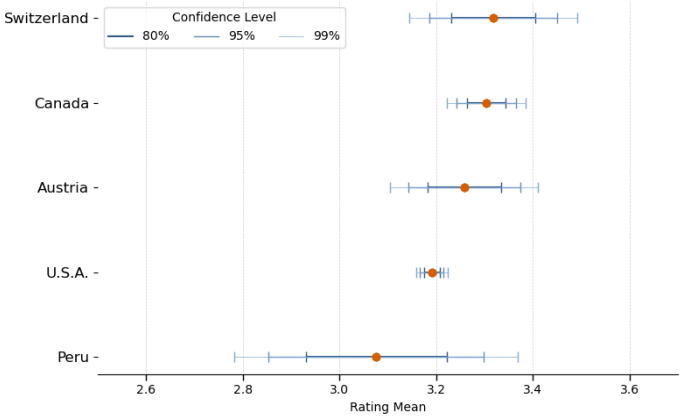
Normal Approximation for Confidence Interval

```
import pandas as pd
import scipy.stats as ss
levels = [0.80, 0.95, 0.99]
levels_df = pd.DataFrame({"level": levels})
cr_CI = cr_mean.assign(key=1).merge(levels_df.assign(key=1), on="key").drop(columns=
    "key")
cr_CI["z"] = cr_CI["level"].apply(lambda lvl: ss.norm.ppf(1 - (1 - lvl) / 2))
cr_CI["conf.low"] = cr_CI["mean"] - cr_CI["z"] * cr_CI["se"]
cr_CI["conf.high"] = cr_CI["mean"] + cr_CI["z"] * cr_CI["se"]
cr_CI["level"] = (cr_CI["level"] * 100).astype(int).astype(str) + "%"
cr_CI = cr_CI[["Company Location", "mean", "se", "level", "conf.low", "conf.high"]]
print(cr_CI)
```

- Usually, we can set different significance levels when constructing confidence intervals

Company	Location	mean	se	level	conf.low	conf.high
	Austria	3.258333	0.059310	80%	3.182324	3.334343
	Austria	3.258333	0.059310	95%	3.142087	3.374580
	Austria	3.258333	0.059310	99%	3.105560	3.411107

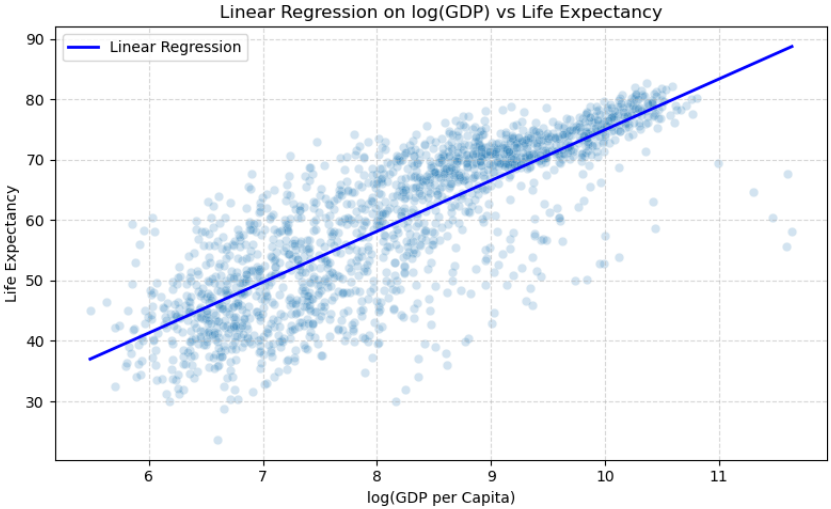
- Confidence interval becomes wider as coverage probability increases.
- What are the common characteristics of these three intervals?



- Confidence interval becomes wider as coverage probability increases.
- What are the common characteristics of these three intervals?

Gapminder Raw Dataset Overview

- **Source**: Gapminder (Hans Rosling team)
- **Coverage**: 142+ countries and regions
- **Variables**:
  - `country`: country name
  - `year`: observation year
  - `pop`: population size
  - `continent`: continent
  - `lifeExp`: life expectancy (years)
  - `gdpPercap`: GDP per capita



- Life Expectancy and log-GDP appears to have linear relationship. We should consider a linear regression model.

Linear Regression Model

Model:

$$Y = aX + b + \epsilon.$$

Assumptions:

$$X \perp \epsilon, \quad \epsilon \sim N(0, \sigma^2).$$

Observed data:

$$(x_i, y_i), \quad i = 1, 2, \ldots, n.$$

Least Squares Estimation:

$$\widehat{a} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}, \quad \widehat{b} = \bar{y} - \widehat{a}\bar{x}.$$

Inference in Linear Regression

Sampling distribution of $\widehat{a}$:

$$\widehat{a} \sim N\left(a,\ \frac{\sigma^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}\right).$$

Prediction at a new point x_0:

$$\widehat{y}_0 = \widehat{a}x_0 + \widehat{b}.$$

Inference in Linear Regression

Confidence interval for the mean response at x_0:

$$\widehat{y}_0 \pm t^*_{n-2}\,\widehat{\sigma}\sqrt{\frac{1}{n} + \frac{(x_0 - \bar{x})^2}{\sum_{i=1}^n (x_i - \bar{x})^2}}.$$

Prediction interval for a new observation at x_0:

$$\widehat{y}_0 \pm t^*_{n-2}\,\widehat{\sigma}\sqrt{1 + \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{\sum_{i=1}^n (x_i - \bar{x})^2}}.$$

Residual standard error:

$$\widehat{\sigma} = \sqrt{\frac{1}{n-2}\sum_{i=1}^n \left(y_i - \widehat{a}x_i - \widehat{b}\right)^2}.$$

Question: Why does prediction interval has additional variance?

Codes for linear regression

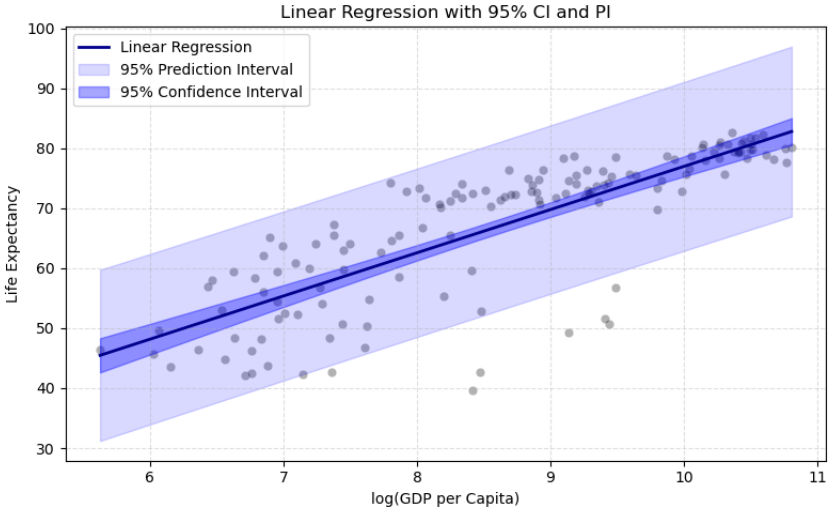
```
df = px.data.gapminder()
df["log_gdp"] = np.log(df["gdpPercap"])
df = df[df["year"] == 2007]
X = df[["log_gdp"]]
y = df["lifeExp"]
model = LinearRegression()
model.fit(X, y)
x_pred = np.linspace(X.min()[0], X.max()[0], 300).reshape(-1, 1)
y_pred = model.predict(x_pred)
X_design = np.hstack([np.ones_like(X), X])
X_pred_design = np.hstack([np.ones_like(x_pred), x_pred])

y_fit = model.predict(X)
residuals = y - y_fit
n = len(y)
mse = np.mean(residuals ** 2)
XtX_inv = np.linalg.inv(X_design.T @ X_design)
t_val = t.ppf(0.975, df=n - 2)
```

Codes for linear regression

```
se_ci = np.sqrt(np.sum((X_pred_design @ XtX_inv) * X_pred_design, axis=1) * mse)
se_pi = np.sqrt((np.sum((X_pred_design @ XtX_inv) * X_pred_design, axis=1) + 1) *
    mse)
y_lower_ci = y_pred - t_val * se_ci
y_upper_ci = y_pred + t_val * se_ci
y_lower_pi = y_pred - t_val * se_pi
y_upper_pi = y_pred + t_val * se_pi

plt.figure(figsize=(8, 5))
sns.scatterplot(x="log_gdp", y="lifeExp", data=df, alpha=0.3, color="black")
plt.plot(x_pred, y_pred, color="darkblue", linewidth=2, label="Linear Regression")
plt.fill_between(x_pred.ravel(), y_lower_pi, y_upper_pi, color="blue", alpha=0.15,
    label="95% Prediction Interval")
plt.fill_between(x_pred.ravel(), y_lower_ci, y_upper_ci, color="blue", alpha=0.35,
    label="95% Confidence Interval")
plt.xlabel("log(GDP per Capita)")
plt.ylabel("Life Expectancy")
plt.title("Linear Regression with 95% CI and PI")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.4)
plt.tight_layout()
```

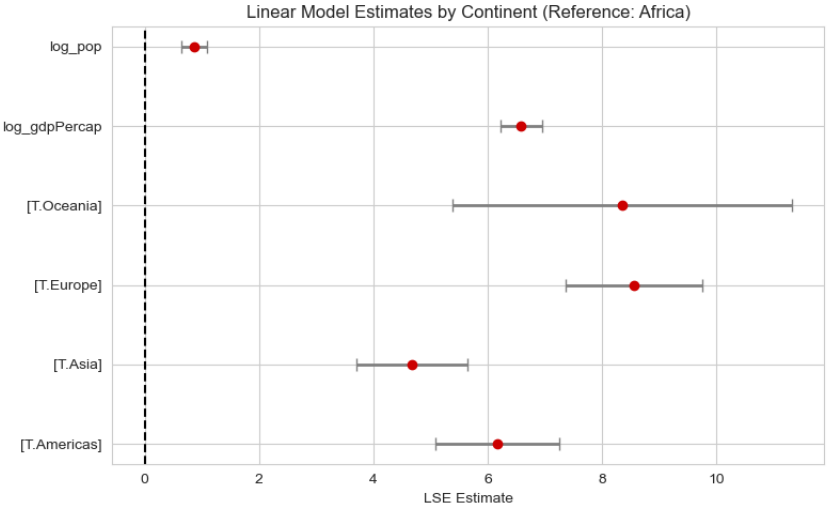


- Prediction interval is significantly wider than the confidence interval.
- Question: Is the upper and lower bounds for intervals are straight?

Multivariate Linear Regression

```
df = px.data.gapminder().copy()
df["log_gdpPercap"] = np.log(df["gdpPercap"])
df["log_pop"] = np.log(df["pop"])
model = smf.ols("lifeExp ~ log_gdpPercap + log_pop + continent", data=df).fit()
summary_df = model.summary2().tables[1]
coefs = model.summary2().tables[1].reset_index().rename(columns={"index": "term"})
conf = model.conf_int().reset_index()
conf.columns = ["term", "conf.low", "conf.high"]
result = pd.merge(coefs, conf, on="term")

result = result[result["term"] != "Intercept"].copy()
result["continent"] = result["term"].str.replace("continent", "", regex=False)
plt.errorbar(
    x=result["Coef."], y=result["continent"],
    xerr=[result["Coef."] - result["conf.low"], result["conf.high"] - result["Coef."
        ]],
    fmt="o", color="#CC0000",
    ecolor="gray", elinewidth=2, capsize=4, markersize=6
)
```



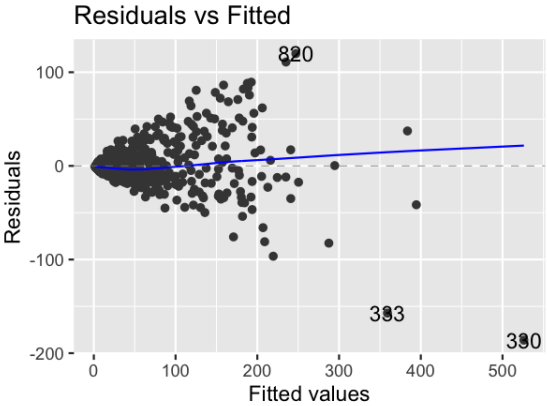
- All coefficients with 95% confidence interval
- Question: Any insignificant predictor?

Model Diagnostic

- Residuals vs Fitted
- Normal Q-Q Plot
- Goal: check model assumptions and fit quality
- Common issues: nonlinearity, heteroscedasticity, non-normal errors

Residuals vs Fitted

- **Purpose**: assess **linearity** and **homoscedasticity**
- Plot residuals against fitted (predicted) values
- A horizontal reference line at 0 indicates zero residuals



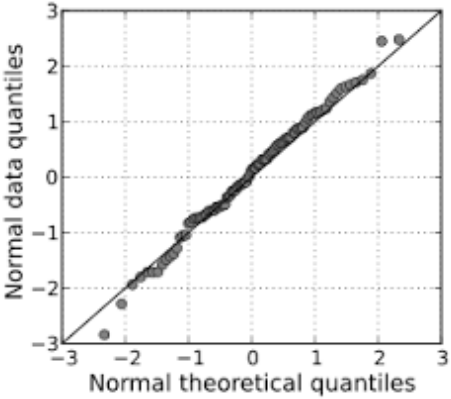
How to Interpret: Residuals vs Fitted

- **Random scatter around 0**: good linear fit, roughly constant variance
- **Curved pattern**: model misspecification / nonlinearity
- **Funnel shape**: heteroscedasticity (non-constant variance)
- **Outliers**: possible influential observations

$$\text{Ideal: } \mathbb{E}[\varepsilon \mid \hat{y}] = 0, \quad \text{Var}(\varepsilon \mid \hat{y}) \approx \text{constant}$$

Normal Q-Q Plot

- **Purpose**: check normality of residuals
- Compares empirical quantiles of residuals with theoretical normal quantiles
- Points close to the diagonal line ⇒ residuals approximately normal



How to Interpret: Normal Q-Q Plot

- **Straight line**: normality assumption is reasonable
- **S-shape**: heavy/light tails
- **Systematic curvature**: skewness
- **Extreme deviations at ends**: outliers

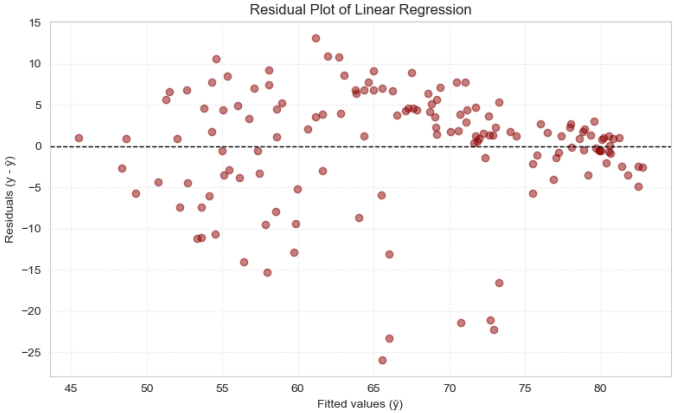
Multivariate Linear Regression

```
df = px.data.gapminder()
df["log_gdp"] = np.log(df["gdpPercap"])
df = df[df["year"] == 2007]
X = df[["log_gdp"]]
y = df["lifeExp"]
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)
residuals = y - y_pred
plt.figure(figsize=(8, 5))
plt.scatter(y_pred, residuals, alpha=0.5, color="darkred")
```

(1) Fit a linear regression

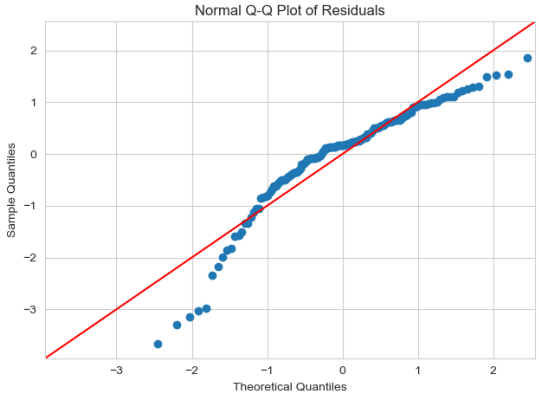
(2) Compute the residuals

(3) Plot fitted values v.s. residuals



- We observe a funnel shape (Heteroskedasticity).

```python
import statsmodels.api as sm
sm.qqplot(residuals/np.std(residuals), line="45")
plt.title("Normal Q-Q Plot of Residuals")
plt.tight_layout()
```



- Can we assume normality?

Normality Test

```
from scipy.stats import shapiro
stat, p = shapiro(residuals)
print(f"Shapiro-Wilk Test: statistic={stat:.4f}, p-value={p:.4f}")
if p > 0.05:
    print("Residuals look normal (fail to reject H0)")
else:
    print("Residuals not normal (reject H0)")
```

- Shapiro-Wilk Test: statistic=0.9003, p-value=0.0000
- Residuals not normal (reject H0)